MODUL 4

Doubly dan Circular Linked List

Modul Praktikum Struktur Data • C++17

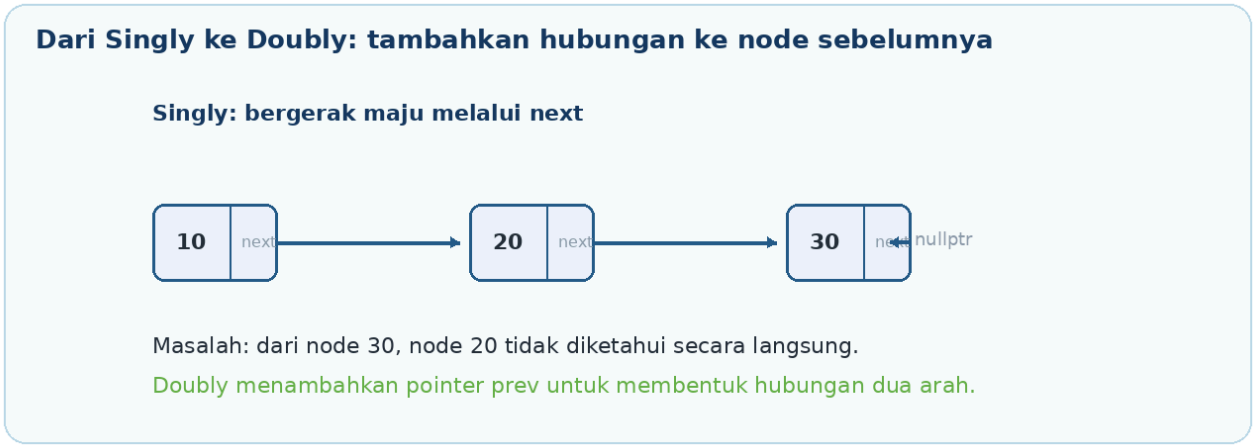
Program Studi Informatika • Praktikum Struktur Data • C++17

1. Tujuan Praktikum

1. menjelaskan keterbatasan singly linked list yang melatarbelakangi penggunaan doubly dan circular linked list;
2. membedakan representasi singly, doubly, dan circular linked list;
3. menjelaskan fungsi prev, next, head, dan tail pada doubly linked list;
4. melakukan traversal doubly linked list dari awal ke akhir dan dari akhir ke awal;
5. menambahkan node pada doubly linked list sambil mempertahankan hubungan antarnode;
6. menghapus node pertama, tengah, terakhir, dan satu-satunya node;
7. menjelaskan hubungan node terakhir dengan node pertama pada circular linked list;
8. melakukan traversal circular linked list tepat satu putaran;
9. menangani list kosong, satu node, dan beberapa node; dan
10. menguji serta membebaskan memori dinamis dengan benar.

2. Dasar Doubly dan Circular Linked List

Pada singly linked list, setiap node hanya menyimpan hubungan menuju node berikutnya. Keterbatasan tersebut menjadi salah satu alasan digunakannya doubly linked list. Pada struktur ini setiap node menyimpan hubungan ke node sebelumnya dan node berikutnya. Circular linked list mengembangkan hubungan dengan cara berbeda: node terakhir tidak menunjuk nullptr, melainkan kembali menunjuk node pertama.



Gambar 1. Dari singly linked list menuju kebutuhan hubungan dua arah.

Dengan demikian, dua perubahan representasi yang dipelajari pada modul ini adalah:

- doubly linked list: menambahkan hubungan ke node sebelumnya; dan
- circular linked list: menghubungkan kembali node terakhir ke node pertama.

2.1 Doubly linked list

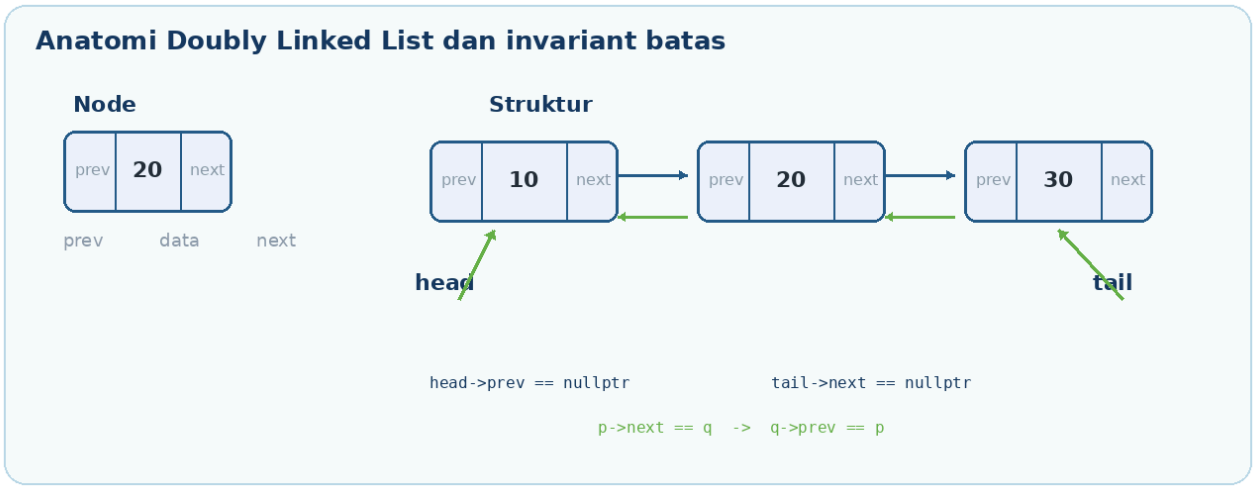
Doubly linked list menggunakan dua pointer pada setiap node: prev menunjuk node sebelumnya dan next menunjuk node berikutnya.

```
struct Node {
    int data;
    Node* prev;
    Node* next;
};
```

Dua pointer lain digunakan untuk mengendalikan struktur:

```
Node* head = nullptr;
Node* tail = nullptr;
```

head menunjuk node pertama, sedangkan tail menunjuk node terakhir.



Gambar 2. Anatomi node, pointer pengendali, dan invariant batas doubly linked list.

2.2 Keadaan doubly linked list

Pada list kosong:

```
head == nullptr
tail == nullptr
```

Jika hanya terdapat satu node:

```
head == tail
head->prev == nullptr
head->next == nullptr
```

Pada beberapa node, node pertama tidak mempunyai pendahulu dan node terakhir tidak mempunyai penerus.

2.3 Invariant doubly linked list

Setelah setiap operasi, hubungan antarnode harus kembali berada pada keadaan yang valid. Kondisi penting yang harus diperiksa adalah:

- head dan tail sama-sama nullptr pada list kosong;
- head->prev == nullptr pada list tidak kosong;
- tail->next == nullptr pada list tidak kosong; dan
- hubungan prev dan next antar-node harus konsisten.

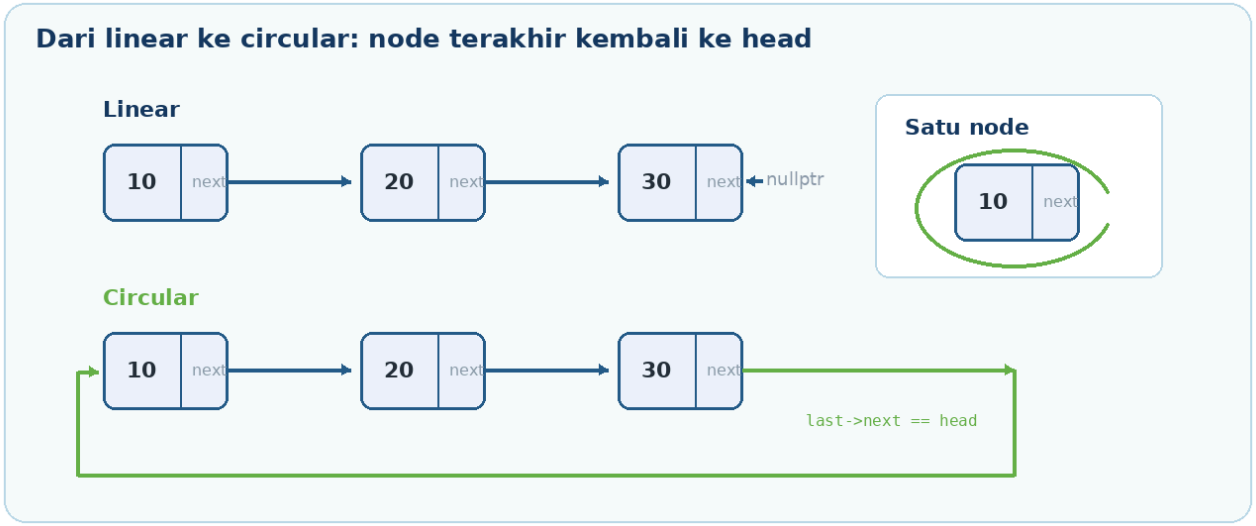
Jika p->next menunjuk q, maka hubungan sebaliknya harus memenuhi q->prev menunjuk p.

```
p->next == q
q->prev == p
```

2.4 Circular linked list

Circular singly linked list menggunakan struktur node yang sama dengan singly linked list. Perbedaannya terletak pada hubungan node terakhir. Jika digunakan pointer last, maka pada list yang tidak kosong harus berlaku:

```
last->next == head
```



Gambar 3. Perubahan hubungan dari linked list linear menjadi circular.

2.5 Keadaan circular linked list

Pada list kosong:

```
head == nullptr
last == nullptr
```

Pada satu node:

```
head == last
head->next == head
```

Pada beberapa node, node terakhir selalu kembali menunjuk head dan tidak ada next anggota list yang bernilai nullptr.

3. Operasi Doubly dan Circular Linked List

3.1 Traversal doubly linked list

Doubly linked list dapat ditelusuri dalam dua arah. Traversal maju dimulai dari head dan mengikuti next, sedangkan traversal mundur dimulai dari tail dan mengikuti prev.

```
void tampilMaju(const Node* head) {
    const Node* bantu = head;
    while (bantu != nullptr) {
        std::cout << bantu->data << ' ';
        bantu = bantu->next;
    }
    std::cout << '\n';
}

void tampilMundur(const Node* tail) {
    const Node* bantu = tail;
    while (bantu != nullptr) {
        std::cout << bantu->data << ' ';
        bantu = bantu->prev;
    }
    std::cout << '\n';
}
```


Gambar 4. Traversal maju menggunakan next dan traversal mundur menggunakan prev.

3.2 Menambah node di akhir doubly linked list

Node baru dibuat dengan kedua pointer awal bernilai nullptr.

```
Node* baru = new Node{nilai, nullptr, nullptr};
```

Jika list kosong, node baru menjadi node pertama sekaligus terakhir.

```
if (head == nullptr) {
    head = tail = baru;
}
```

Jika list tidak kosong, pasang hubungan antara tail lama dan node baru, kemudian pindahkan tail.

```
baru->prev = tail;
tail->next = baru;
tail = baru;
```


Gambar 5. Penambahan node pada akhir doubly linked list.

3.3 Menghapus node doubly linked list

Penghapusan harus memperbaiki hubungan antarnode sebelum memori node dibebaskan. Untuk node tengah, dua hubungan perlu diperbaiki.

```
target->prev->next = target->next;
target->next->prev = target->prev;
delete target;
```

Gambar 6. Penghapusan node memperbaiki hubungan sebelum delete.

3.4 Kasus batas penghapusan

Penghapusan harus mempertimbangkan posisi node: pertama, tengah, terakhir, dan satu-satunya node.

Jika node yang dihapus adalah head:

```
head = target->next;
if (head != nullptr)
    head->prev = nullptr;
```

Jika node yang dihapus adalah tail:

```
tail = target->prev;
if (tail != nullptr)
    tail->next = nullptr;
```

Jika node merupakan satu-satunya node, head dan tail menjadi nullptr setelah node dibebaskan.

3.5 Traversal circular linked list

Traversal circular tidak dapat menggunakan nullptr sebagai kondisi berhenti karena pointer akan kembali ke node awal. Traversal satu putaran dapat ditulis sebagai berikut.

```
void tampilCircular(const NodeCircular* head) {
    if (head == nullptr)
        return;

    const NodeCircular* bantu = head;
    do {
        std::cout << bantu->data << ' ';
        bantu = bantu->next;
    } while (bantu != head);

    std::cout << '\n';
}
```


Gambar 7. Traversal circular berhenti setelah pointer kembali ke node awal.

3.6 Membentuk circular linked list sederhana

```
NodeCircular* head = nullptr;
NodeCircular* last = nullptr;
```

Node pertama dibuat sehingga next kembali menunjuk head.

```
NodeCircular* baru = new NodeCircular{10, nullptr};
head = last = baru;
baru->next = head;
```

Untuk menambahkan node berikutnya:

```
NodeCircular* baru = new NodeCircular{20, nullptr};
baru->next = head;
last->next = baru;
last = baru;
```

Setelah setiap penambahan, periksa last->next == head.

3.7 Pengayaan: circular doubly linked list

Sifat doubly dan circular dapat digabungkan. Setiap node mempunyai prev dan next, sedangkan node pertama dan terakhir juga saling terhubung. Circular doubly linked list pada modul ini hanya digunakan untuk menunjukkan kemungkinan kombinasi representasi; implementasinya tidak menjadi kegiatan praktikum.

Gambar 8. Konsep circular doubly linked list sebagai pengayaan.

4. Kegiatan Praktikum

Kerjakan kegiatan secara berurutan. Kompilasi program setelah setiap perubahan.

4.1 Doubly linked list dasar

Simpan program dalam latihan_dll.cpp.

1. definisikan struct Node yang mempunyai data, prev, dan next;
2. buat head dan tail dengan nilai awal nullptr;
3. buat fungsi tambahAkhir;
4. masukkan data 10, 20, dan 30;
5. tampilkan data dari head menuju tail;
6. tampilkan data dari tail menuju head; dan
7. periksa head->prev == nullptr dan tail->next == nullptr.

Hasil traversal maju: 10 20 30. Hasil traversal mundur: 30 20 10.

4.2 Penghapusan node

Tambahkan fungsi berikut:

```
bool hapusData(Node*& head, Node*& tail, int nilai);
```

1. hapus node tengah;
2. hapus node pertama;
3. hapus node terakhir;
4. hapus satu-satunya node; dan

5. hapus data yang tidak tersedia.

Setelah setiap operasi, tampilkan list maju dan mundur, kemudian periksa kembali hubungan head, tail, prev, dan next.

4.3 Circular linked list sederhana

Simpan latihan dalam latihan_circular.cpp.

1. buat struktur NodeCircular;
2. buat pointer head dan last;
3. tambahkan data 10, 20, dan 30;
4. pastikan node terakhir kembali menunjuk head;
5. tampilkan seluruh data tepat satu putaran; dan
6. ulangi pengujian menggunakan hanya satu node.

```
last->next == head

// satu node
head == last
head->next == head
```

5. Tugas Praktikum

Buat program Playlist Lagu menggunakan doubly linked list. Simpan seluruh program dalam modul4_playlist.cpp.

Setiap lagu mempunyai judul dan nama penyanyi. Program harus dapat:

1. menambahkan lagu pada akhir playlist;
2. menolak judul lagu yang sudah tersedia;
3. menampilkan playlist dari awal ke akhir;
4. menampilkan playlist dari akhir ke awal;
5. mencari lagu berdasarkan judul;
6. menghapus lagu berdasarkan judul;
7. menampilkan jumlah lagu; dan
8. membebaskan seluruh node sebelum program berakhir.

5.1 Kerangka program

```
#include <iostream>
#include <string>

struct Lagu {
    std::string judul;
    std::string penyanyi;
};

struct Node {
    Lagu data;
    Node* prev;
    Node* next;
```

```
};

bool judulTersedia(const Node* head, const std::string& judul);
bool tambahAkhir(Node*& head, Node*& tail, const Lagu& data);
Node* cariJudul(Node* head, const std::string& judul);
bool hapusJudul(Node*& head, Node*& tail, const std::string& judul);
void tampilMaju(const Node* head);
void tampilMundur(const Node* tail);
int jumlahData(const Node* head);
void clear(Node*& head, Node*& tail);
```

Fungsi main menggunakan menu sederhana berikut.

```
int main() {
    Node* head = nullptr;
    Node* tail = nullptr;
    int pilihan;

    do {
        std::cout << "\n1. Tambah lagu\n";
        std::cout << "2. Tampilkan maju\n";
        std::cout << "3. Tampilkan mundur\n";
        std::cout << "4. Cari lagu\n";
        std::cout << "5. Hapus lagu\n";
        std::cout << "6. Jumlah lagu\n";
        std::cout << "7. Keluar\n";
        std::cout << "Pilihan: ";
        std::cin >> pilihan;

        // panggil fungsi sesuai pilihan

    } while (pilihan != 7);

    clear(head, tail);
    return 0;
}
```

6. Pengujian

Jalankan seluruh skenario berikut. Catat hasil aktual dan perbaiki program jika hasilnya tidak sesuai.

No.	Skenario	Hasil yang diharapkan
1	Tampilkan DLL kosong	Tidak ada data
2	Tambah lagu pertama	head == tail
3	Periksa satu node	head->prev == nullptr dan tail->next == nullptr
4	Tambah beberapa lagu	Urutan maju benar
5	Tampilkan mundur	Urutan terbalik benar
6	Hapus lagu pertama	head diperbarui dan head->prev == nullptr
7	Hapus lagu tengah	Hubungan prev dan next diperbaiki

8	Hapus lagu terakhir	tail diperbarui dan tail->next == nullptr
9	Hapus satu-satunya lagu	head dan tail menjadi nullptr
10	Hapus judul tidak tersedia	List tidak berubah
11	Circular kosong	head dan last nullptr
12	Circular satu node	head == last dan head->next == head
13	Circular beberapa node	last->next == head
14	Traversal circular	Setiap node tampil tepat satu kali
15	clear playlist	Seluruh node DLL dibebaskan

6.1 Diagnosis kode

Setiap potongan berikut mengandung kesalahan hubungan pointer atau pengelolaan memori. Tentukan kesalahan, akibat, dan perbaikannya.

A. prev node baru tidak dipasang

```
tail->next = baru;
tail = baru;
```

B. Hanya hubungan maju yang diperbaiki

```
target->prev->next = target->next;
delete target;
```

C. tail dihapus sebelum diperbarui

```
delete tail;
```

D. Node tidak dibebaskan

```
head = nullptr;
tail = nullptr;
```

E. Traversal circular menunggu nullptr

```
NodeCircular* bantu = head;
while (bantu != nullptr) {
    bantu = bantu->next;
}
```

F. Node terakhir tidak kembali ke head

```
last->next = baru;
last = baru;
baru->next = nullptr;
```

G. Circular satu node dibentuk seperti list linear

```
head = baru;
last = baru;
baru->next = nullptr;
```